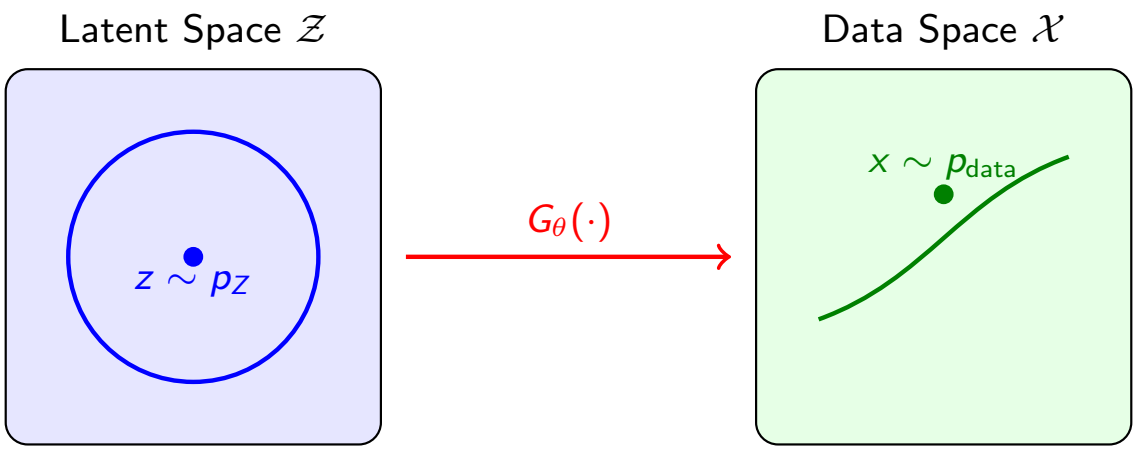
Intro to Generative Modeling

Machine Learning Journal Club, OSU

March 23, 2026

The Problem: Pushforward Maps

- **Given:** A simple, known latent distribution $Z \sim p_Z$ (e.g., $\mathcal{N}(0, I)$).
- **Target:** A complex, unknown data distribution $X \sim p_{\text{data}}$ on a manifold $\mathcal{M}$.
- **Goal:** Find a deterministic map $G_\theta : \mathcal{Z} \to \mathcal{X}$ such that the *pushforward* of p_Z approximates p_{data}.
- Formally, we want $p_\theta = G_{\theta\sharp} p_Z \approx p_{\text{data}}$.



Why Neural Networks?

Why not polynomials, splines, or Fourier series?

Approximating arbitrary pushforward maps in high-dimensional spaces (e.g., images, audio) suffers heavily from the **curse of dimensionality**.

Advantages of Deep Neural Networks (DNNs):

- **High-Dimensional Scalability:** Parameterized neural architectures exploit structural priors (translation invariance, locality) that circumvent traditional grid-based scaling limits.
- **Universal Approximation:** Wide or deep networks can represent essentially any Borel measurable function G.
- **Compositionality:** Deep layers allow hierarchical manifold folding, which warps a simple Gaussian into a highly complex data manifold step-by-step.

Generative Adversarial Networks (GANs)

- Instead of explicit density estimation, GANs define a two-player minimax game.
- **Generator (G_θ):** Learns the pushforward map $X = G_\theta(Z)$.
- **Discriminator (D_ϕ):** Learns to distinguish between true samples $x \sim p_{\text{data}}$ and generated samples $\hat{x} \sim G_{\theta\sharp} p_Z$.

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

Wasserstein GAN (WGAN) Derivation

Goal: Minimize the Wasserstein-1 distance between the true data distribution and our pushforward distribution:

$$\min_{G_\theta} W(p_{\text{data}}, G_{\theta\sharp} p_Z)$$

Primal Form: The cost of the optimal transport plan γ:

$$W(p_{\text{data}}, p_\theta) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_\theta)} \mathbb{E}_{(x,y)\sim\gamma}[||x - y||]$$

Kantorovich Duality: Converts the intractable infimum over joint distributions into a supremum over 1-Lipschitz functions f:

$$W(p_{\text{data}}, p_\theta) = \sup_{||f||_L \leq 1} \mathbb{E}_{x\sim p_{\text{data}}}[f(x)] - \mathbb{E}_{y\sim p_\theta}[f(y)]$$

Final Objective: Substituting $y = G_\theta(z)$, we get our minimax game:

$$\min_{G_\theta} \max_{||f||_L \leq 1} \mathbb{E}_{x\sim p_{\text{data}}}[f(x)] - \mathbb{E}_{z\sim p_Z}[f(G_\theta(z))]$$

Variational Autoencoders (VAEs)

- Unlike GANs, VAEs optimize an explicit likelihood, but use variational inference to handle intractable integrals.
- **Encoder (Inference):** $q_\phi(z|x) \approx p(z|x)$
- **Decoder (Pushforward):** $p_\theta(x|z)$

The Objective: Maximize the Evidence Lower Bound (ELBO):

$$\log p(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}(q_\phi(z|x)||p_Z(z))$$

- This balances reconstruction with latent space regularization.
- It creates a continuous, smooth latent space, though it often produces blurrier images compared to GANs due to the Gaussian likelihood assumptions.

Deriving the ELBO

We want to maximize the log-likelihood of the data, $\log p(x)$:

$$\begin{aligned}
\log p(x) &= \log \int p(x, z) dz \\
&= \log \int q_\phi(z|x) \frac{p(x, z)}{q_\phi(z|x)} dz \\
&= \log \mathbb{E}_{q_\phi(z|x)} \left[\frac{p(x, z)}{q_\phi(z|x)} \right]
\end{aligned}$$

Applying Jensen's Inequality ($\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$):

$$\begin{aligned}
\log p(x) &\geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x|z) p_Z(z)}{q_\phi(z|x)} \right] \\
&= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] + \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_Z(z)}{q_\phi(z|x)} \right] \\
&= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}(q_\phi(z|x) || p_Z(z))
\end{aligned}$$

Normalizing Flows and The Continuity Equation

Instead of a single discrete pushforward map, what if we smoothly transform the density over time $t \in [0, 1]$?

- Let $p_t(x)$ be a time-varying probability density, where $p_0 = p_Z$ and $p_1 = p_{\text{data}}$.
- The evolution of the density is governed by a vector field $v_t(x)$ via the **Continuity Equation**:

$$\frac{\partial}{\partial t} p_t(x) + \nabla \cdot (p_t(x) v_t(x)) = 0$$

- If we can learn $v_t(x)$, we can generate data by solving the ODE: $\frac{dx}{dt} = v_t(x)$ starting from $x_0 \sim p_Z$.

Conditional Flow Matching (CFM)

Challenge: Learning the marginal vector field $v_t(x)$ directly is intractable.
Solution (CFM): Define a *conditional* vector field $u_t(x|x_1)$ that pushes p_Z to a specific data point x_1.

- By marginalizing over the data, the true vector field is exactly recovered:

$$v_t(x) = \int u_t(x|x_1) \frac{p_t(x|x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1$$

- We train a neural network $v_\theta(x, t)$ to match the conditional vector fields via a simple MSE loss:

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t, x_1, x|x_1} \left[||v_\theta(x, t) - u_t(x|x_1)||^2 \right]$$

Deriving the Marginal Vector Field (Part 1)

The Premise: We know that the conditional vector field $u_t(x|x_1)$ generates the conditional probability path $p_t(x|x_1)$.
Therefore, it perfectly satisfies the *conditional* continuity equation:

$$\frac{\partial}{\partial t} p_t(x|x_1) + \nabla \cdot (p_t(x|x_1) u_t(x|x_1)) = 0$$

The Goal: We want to find a marginal vector field $v_t(x)$ that satisfies the *marginal* continuity equation:

$$\frac{\partial}{\partial t} p_t(x) + \nabla \cdot (p_t(x) v_t(x)) = 0$$

where $p_t(x) = \int p_t(x|x_1) p_{\text{data}}(x_1) dx_1$.

Deriving the Marginal Vector Field (Part 2)

Let us marginalize the conditional continuity equation by integrating it over the true data distribution $p_{\text{data}}(x_1)$:

$$\int \left[\frac{\partial}{\partial t} p_t(x|x_1) + \nabla \cdot (p_t(x|x_1) u_t(x|x_1)) \right] p_{\text{data}}(x_1) dx_1 = 0$$

By the linearity of integration and differentiation, we can pull the derivatives outside the integral:

$$\frac{\partial}{\partial t} \underbrace{\int p_t(x|x_1) p_{\text{data}}(x_1) dx_1}_{p_t(x)} + \nabla \cdot \int p_t(x|x_1) u_t(x|x_1) p_{\text{data}}(x_1) dx_1 = 0$$

Deriving the Marginal Vector Field (Part 3)

Substituting $p_t(x)$ into our previous result, we get:

$$\frac{\partial}{\partial t} p_t(x) + \nabla \cdot \left(\int p_t(x|x_1) u_t(x|x_1) p_{\text{data}}(x_1) dx_1 \right) = 0$$

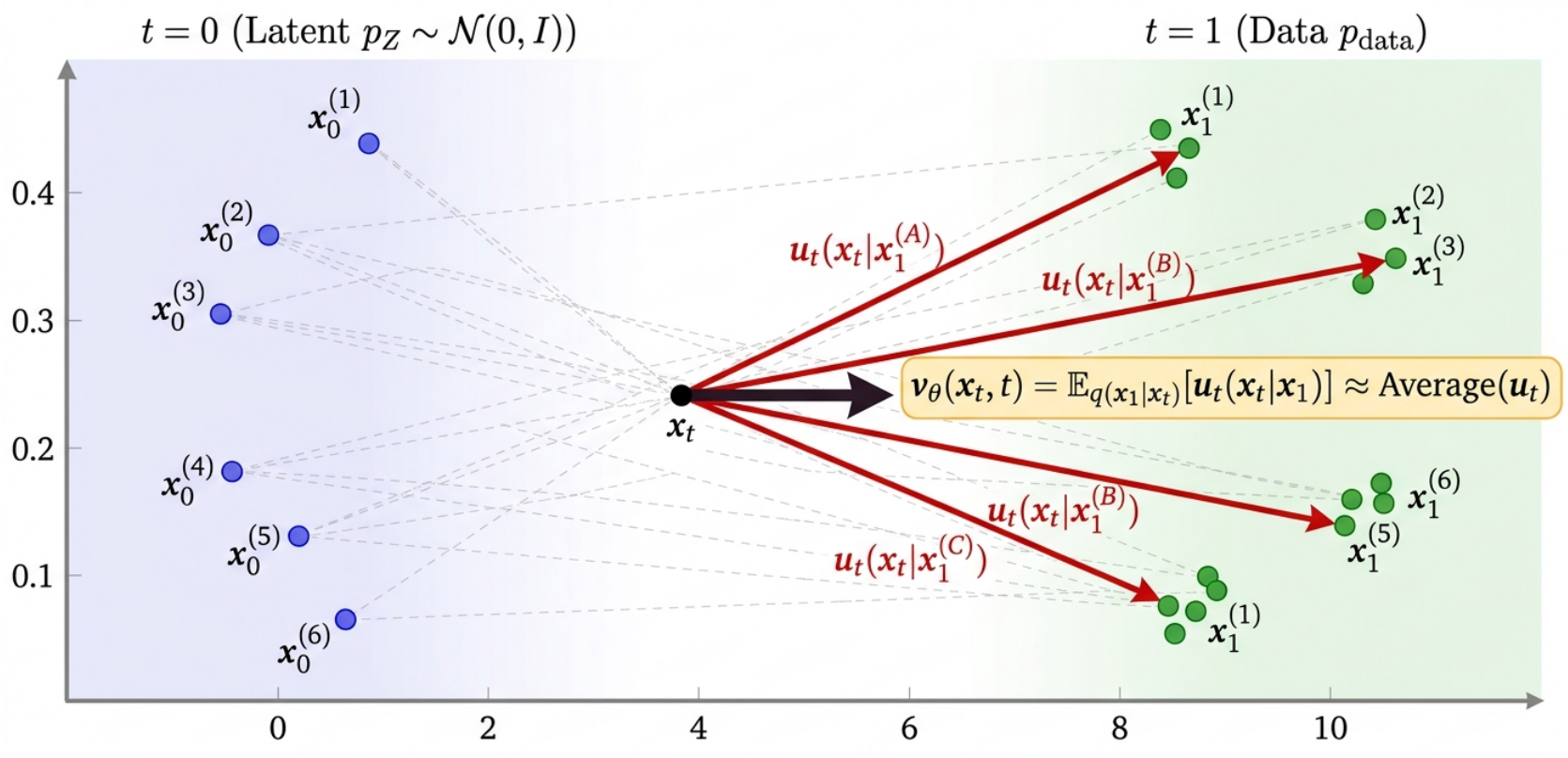
To match the marginal continuity equation $\frac{\partial}{\partial t} p_t(x) + \nabla \cdot (p_t(x) v_t(x)) = 0$, we simply equate the terms inside the divergence operator:

$$p_t(x) v_t(x) = \int p_t(x|x_1) u_t(x|x_1) p_{\text{data}}(x_1) dx_1$$

Dividing both sides by $p_t(x)$ gives our exact formulation for CFM:

$$v_t(x) = \int u_t(x|x_1) \frac{p_t(x|x_1) p_{\text{data}}(x_1)}{p_t(x)} dx_1$$

Visualizing Conditional Flow Matching



Text Generation: The Goal & The Old Way

The Core Goal: At its heart, AI text generation is simply playing a massive game of "predict the next word" (or token) based on the context of the previous words.

The Pre-Transformer Era:

- Older models (like RNNs and LSTMs) processed text sequentially, reading one word at a time from left to right.
- **The Bottleneck:** Because they read like humans, they had a "memory" problem. By the time they reached the end of a long paragraph, they often forgot the context from the very first sentence.
- This made generating long, coherent text incredibly difficult.

The Transformer Leap: Parallel Processing

In 2017, the paper *Attention Is All You Need* introduced the **Transformer** architecture, completely changing how machines read.

The Paradigm Shift:

- **Looking Everywhere at Once:** Instead of reading left-to-right, the Transformer takes in *every single word* of the input sequence simultaneously.
- **No More Forgetting:** Because all words are processed in parallel, the distance between the first word and the last word doesn't matter anymore.
- This parallel approach relies entirely on a mechanism called **Self-Attention**.

Self-Attention

Self-Attention allows the model to look at a word and ask: *"Which other words in this sentence help me understand the true meaning of this specific word?"*

The Homonym Example:

- Sentence A: *"The* **bank** *of the river was steep."*
- Sentence B: *"I deposited money in the* **bank***."*

In Sentence A, the Self-Attention mechanism assigns high "attention weights" between **bank**, **river**, and **steep**. The model dynamically updates its internal definition of "bank" based on its neighbors, completely disambiguating the word.

How Attention Works: A Simple Analogy

To figure out which words matter to each other, the model uses a system of **Queries, Keys, and Values** (like a library search):

- **Query (What I'm looking for):** The current word asks a question. (e.g., "I am the word 'bank'. Are there any water-related words around?")
- **Key (What I have):** Every other word holds a label. (e.g., The word 'river' says "I am a water-related noun.")
- **Value (The meaning):** If the Query and Key match, the actual context/meaning of the Value is absorbed by the original word.

Through this process, every word becomes a rich, context-aware combination of the entire sentence.

Putting it Together: Autoregressive Generation

From Understanding to Writing: Once the Transformer has used Self-Attention to deeply understand the context of the prompt, how does it write?

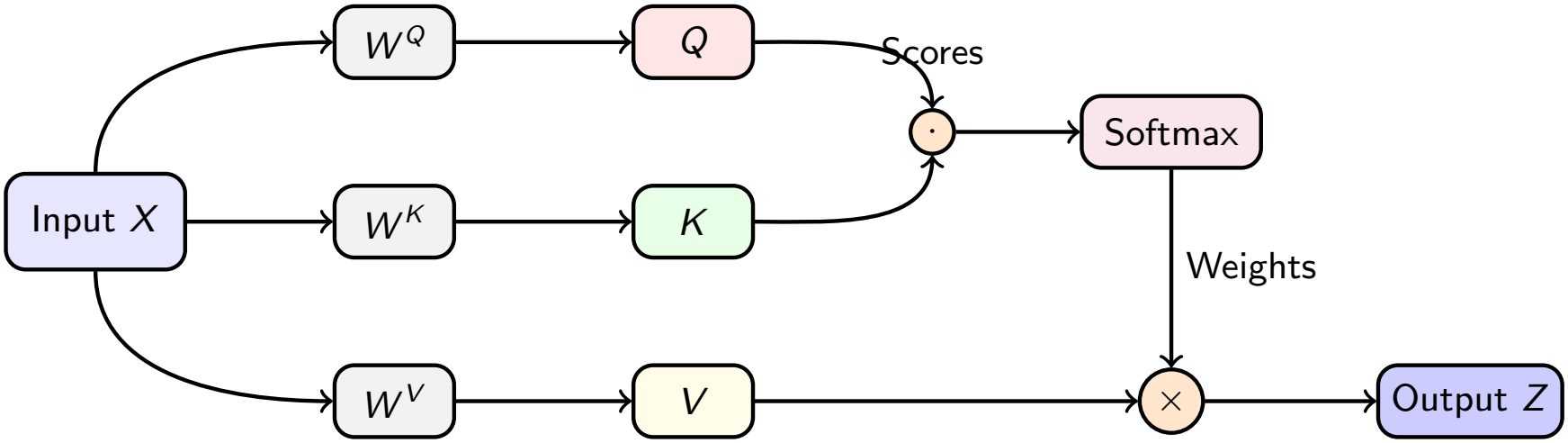
1. It analyzes all the words currently in the sequence.
2. It uses its vast training data to calculate the probability of what the *very next* word should be.
3. It selects a word, appends it to the sequence, and **repeats the entire process**.

By relying on perfect, attention-driven memory, modern AI can generate highly coherent, contextually accurate paragraphs, one calculated word at a time.

Visualizing Self-Attention: Queries, Keys, and Values

The Information Retrieval Analogy:

- **Query (Q):** What is this word looking for? (Search term)
- **Key (K):** What information does this word hold? (Index or Title)
- **Value (V):** What is the actual content of this word? (Search result)



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Open Problems in Generative Modeling

Despite incredible progress, generative AI still faces significant hurdles:

- **Evaluation Metrics:** How do we objectively measure the "creativity" or "quality" of an image or text? Current metrics (like FID for images) are flawed and often don't align with human perception.
- **Mode Collapse & Diversity:** Models often find a few "safe" outputs that fool the objective function and repeatedly generate them, ignoring the full diversity of the real training data.
- **Inference Speed vs. Quality:** Diffusion and Flow models produce stunning quality but require many sequential steps to generate a single sample, making them computationally expensive at inference time.

Readings

If you want to dive deeper into the foundations, here are the papers that defined the modern era of generative modeling:

- **GANs:** *Generative Adversarial Nets* (Goodfellow et al., 2014).
- **VAEs:** *Auto-Encoding Variational Bayes* (Kingma & Welling, 2013).
- **Transformers:** *Attention Is All You Need* (Vaswani et al., 2017). The architecture that replaced RNNs and birthed modern Large Language Models.
- **Diffusion:** *Denoising Diffusion Probabilistic Models* (Ho et al., 2020). The paper that made diffusion models practical for high-fidelity image generation.
- **CFM:** *Flow Matching for Generative Modeling* (Lipman et al., 2022). The framework that unifies Continuous Normalizing Flows and Diffusion through vector field regression.